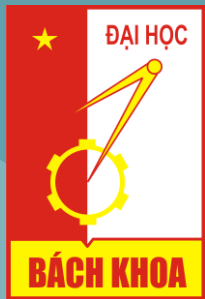


C PROGRAMMING INTRODUCTION



TUẦN 4: BIẾN, HẲNG VÀ ĐẦU VÀO CHUẨN

**Trường Công nghệ Thông tin và Truyền thông
Đại học Bách Khoa Hà Nội**

Nội dung

- Biến
 - Ôn tập lý thuyết
 - Biến
 - Các kiểu dữ liệu chuẩn
 - Hằng số
 - Đầu vào chuẩn
 - Bài tập thực hành

Định danh

- Tên của các đối tượng (biến, hàm, v.v)

// biến

*int **nMyPresentIncome** = 0;*

// khai báo nguyên mẫu hàm

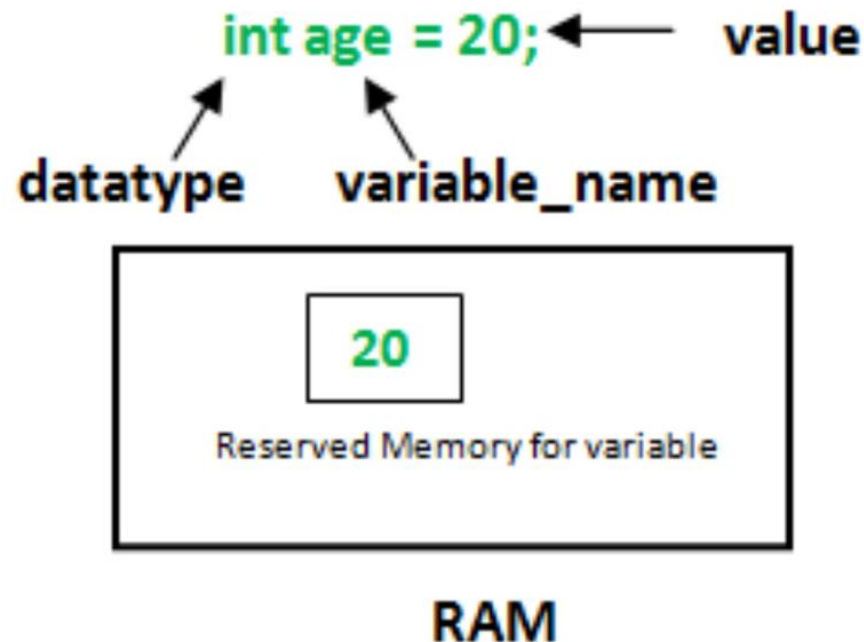
*int **DownloadOrBuyCD**();*

Quy tắc đặt tên

- Chữ cái, chữ số, dấu gạch dưới ‘_’
 - `i`
 - `CSE_5a`
 - `a_very_long_name_that_isnt_very_useful`
 - `fahrenheit`
- Không bắt đầu bằng chữ số
 - `5a_CSE` : không hợp lệ
- Phân biệt hoa thường
 - `CSE_5a` khác `cse_5a`

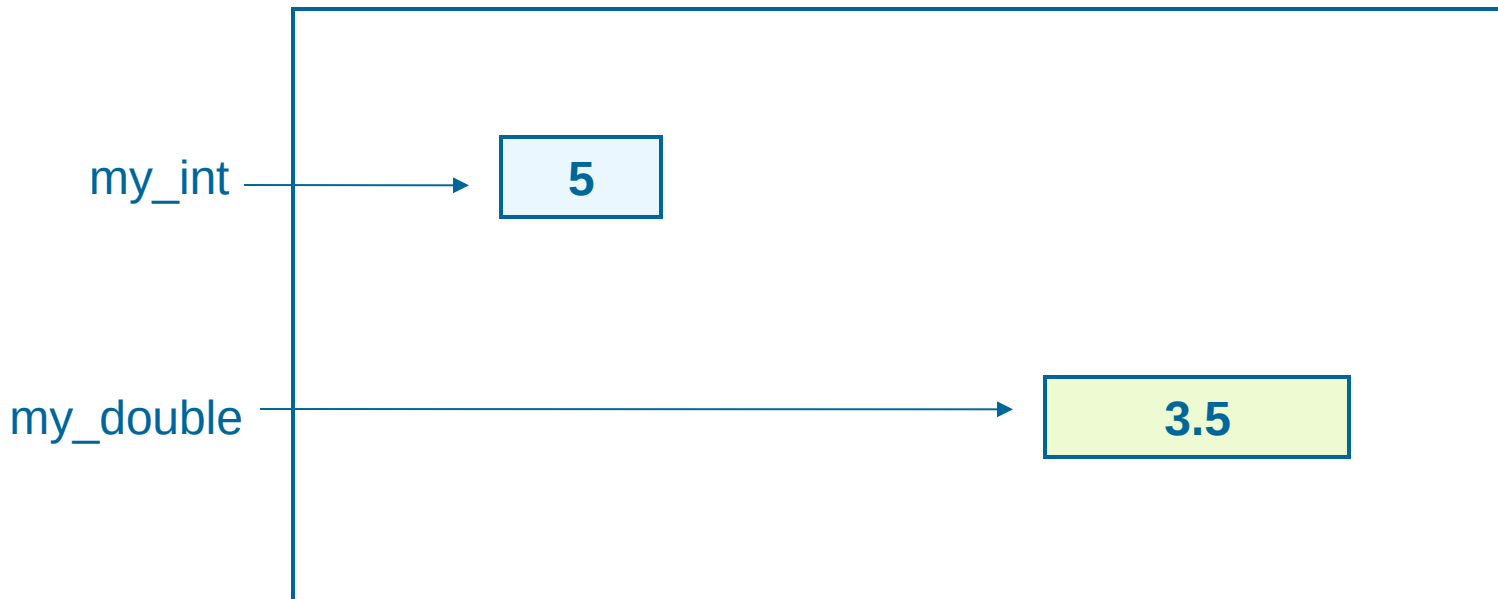
Biến là gì?

- Một vùng nhớ được đặt tên trong máy tính sử dụng để chứa dữ liệu ở một dạng nào đó: số nguyên, số thực, v.v
- Chứa dữ liệu đang được chương trình sử dụng
- Tóm lại, biến là cách duy nhất để thao tác với dữ liệu



Biến trong bộ nhớ

```
int my_int = 5;  
double my_double = 3.5;
```



Khai báo, định nghĩa và khởi tạo

- Khai báo cung cấp vùng lưu trữ gọi là định nghĩa biến:
`int j;`
- Định nghĩa biến: có thể gán giá trị cho biến (khởi tạo)
`int j = 0;`
- Chú ý: Các bản phát hành thường không khởi tạo biến bởi giá trị mặc định
- Định nghĩa biến: có thể mô tả ý nghĩa biến nhưng không cung cấp vùng lưu trữ (vd *extern*)
`extern int j;`
- Ví dụ sử dụng biến:
`printf(“%d + %d = %d\n“, a, b, c);`

Ví dụ: Khai báo biến

- `int i;`
- `char c;`
- `float f1, f2;`
- `float f3 = 7.0, f4 = 5.2;`
- `unsigned int ui = 0;`

Ví dụ: Tính tổng hai số

```
1. #include <stdio.h>
2.
3. int main()
4. {
5.     int a, b, c;
6.     printf("The first number: ");
7.     scanf("%d", &a);
8.     printf("The second number: ");
9.     scanf("%d", &b);
10.    c = a + b;
11.    printf("%d + %d = %d\n", a, b, c);
12.    return 0;
13. }
```

12	c
7	b
5	a

\$ /ngonnguC/bin/tong

The first number: 5

The second number: 7

5 + 7 = 12

\$ /ngonnguC/bin/

Biến số

- Sử dụng để *lưu trữ giá trị* và giá trị này *có thể thay đổi*
- Khai báo: **kiểu_biến** *tên_biến*;
- Ví dụ:
int i;
float x, y, z;
char c;
- Phép gán: <tên_biến> = <giá_trị>;
Ví dụ:
i = 4;
x = 5.4;
y = z = 1.2;

Hằng số

- Hằng số: có giá trị không đổi trong suốt chương trình
- Khai báo hằng:

#define *tên_hằng* *giá_trị*

- Ví dụ:

```
#define TRUE 1
```

```
#define FALSE 0
```

Hằng số (1)

- Hằng số nguyên

```
31          /* decimal */  
037         /* octal */  
0x1F        /* hexadecimal */  
31L         /* long */  
31LU        /* unsigned long */
```

- Hằng số thực

```
123.4       /* double */  
123.4F      /* float */  
123.        /* double */  
123.F       /* float */  
123.4L      /* long double */  
1e-2        /* double */  
123.4e-3    /* double */
```

Hàng số (2)

- Hàng kí tự

```
'K'          /* normal ASCII - 'K' */
'\113'       /* octal ASCII - 'K' */
'\x48'       /* hexadecimal ASCII - 'K' */
'\n'         /* normal ASCII - newline */
'\t'         /* normal ASCII - tab */
'\\'         /* normal ASCII - backslash */
'\\"'        /* normal ASCII - double quote */
'\0'         /* normal ASCII - null (marks end of string)*/
```

- Hàng chuỗi

```
"You have fifteen thousand new messages."
"I said, \"Crack, we're under attack!\"."
"hello," "world" trở thành → "hello, world"
```

Các kiểu dữ liệu cơ sở (1)

- Kích thước và giới hạn (có thể thay đổi tùy theo máy) :

<i>Kiểu dữ liệu</i>	<i>Kích thước (bit)</i>	<i>Giới hạn</i>
char	8	-128 ... 127
short	16	-32,768 ... 32,767
int	32	-2,147,483,648 ... 2,147,483,647
long	32	-2,147,483,648 ... 2,147,483,647
float	32	10^{-38} ... 3×10^{38}
double	64	10^{-308} ... 10^{308}

Các kiểu dữ liệu cơ sở (2)

- Bao gồm các giá trị không dấu:

<i>Kiểu dữ liệu</i>	<i>Kích thước (bit)</i>	<i>Giới hạn</i>
unsigned char	8	0 ... 255
unsigned short	16	0 ... 65,535
unsigned int	32	0 ... 4,294,967,295
unsigned long	32	0 ... 4,294,967,295

Định dạng đầu vào với **scanf** (1)

- Định dạng

scanf(chuỗi_điều_khiển_định_dạng [, các_tham_số]);

- chuỗi_điều_khiển_định_dạng : mô tả định dạng đầu vào
- các_tham_số : con trỏ tới các biến chứa giá trị dữ liệu
- có thể bao gồm các độ rộng trường để đọc một số lượng kí tự xác định từ luồng

Định dạng đầu vào với `Scanf` (2)

Conversion specifier	Description
<i>Integers</i>	
d	Read an optionally signed decimal integer. The corresponding argument is a pointer to integer.
i	Read an optionally signed decimal, octal, or hexadecimal integer. The corresponding argument is a pointer to integer.
o	Read an octal integer. The corresponding argument is a pointer to unsigned integer.
u	Read an unsigned decimal integer. The corresponding argument is a pointer to unsigned integer.
x or X	Read a hexadecimal integer. The corresponding argument is a pointer to unsigned integer.
h or l	Place before any of the integer conversion specifiers to indicate that a short or long integer is to be input.
<i>Floating-point numbers</i>	
e , E , f , g or G	Read a floating-point value. The corresponding argument is a pointer to a floating-point variable.
l or L	Place before any of the floating-point conversion specifiers to indicate that a double or long double value is to be input.
<i>Characters and strings</i>	
c	Read a character. The corresponding argument is a pointer to char , no null (<code>'\0'</code>) is added.
s	Read a string. The corresponding argument is a pointer to an array of type char that is large enough to hold the string and a terminating null (<code>'\0'</code>) character—which is automatically added.
<i>Scan set</i>	
<i>[scan characters</i>	Scan a string for a set of characters that are stored in an array.
<i>Miscellaneous</i>	
p	Read an address of the same form produced when an address is output with %p in a printf statement.
n	Store the number of characters input so far in this scanf . The corresponding argument is a pointer to integer
%	Skip a percent sign (<code>%</code>) in the input.

Bài thực hành 1a

- Viết hoàn chỉnh chương trình, bổ sung các câu lệnh hiển thị các kết quả vừa nhập.
- Thử nghiệm với các giá trị nhập vào 4, 22062007, 22/06/2007, Ab, 2.3. Ghi lại kết quả thu được.

```
int d,m,y,x;  
char ch1,ch2;  
float f;  
scanf ("%d", &x) ;  
  
scanf ("%2d%2d%4d", &d, &m, &y) ;  
  
scanf ("%d/%d/%d", &d, &m, &y) ;  
  
scanf ("%c%c", &ch1, &ch2) ;  
  
scanf ("%f", &f) ;
```

Bài thực hành 1b

- Nhập xâu ký tự sau khi nhập số

- Ghi lại kết quả của các chương trình sau

```
// Code 1b_a
#include <stdio.h>
#define MAX_LENGTH 40
int main() {
    char str[MAX_LENGTH];
    int n;

    scanf("%d", &n);
    scanf("%s", str);

    printf("%s\n", str);
    printf("%d\n", n);
    return 0;
}
```

```
//Code 1b_b
#include <stdio.h>
#define MAX_LENGTH 40
int main() {
    char str[MAX_LENGTH];
    int n;

    scanf("%d", &n);
    fgets(str, MAX_LENGTH, stdin);

    printf("%s\n", str);
    printf("%d\n", n);
    return 0;
}
```

Bài thực hành 1b

```
// Code 1b_c
#include <stdio.h>
int main() {
    char str[MAX_LENGTH];
    int n;

    scanf("%d\n", &n);

    fgets(str, MAX_LENGTH, stdin);

    printf("%s\n", str);
    printf("%d\n", n);
    return 0;
}
```

```
// Code 1b_d
#include <stdio.h>
#include <string.h>
int main() {
    char str[MAX_LENGTH];
    int n;

    scanf("%d\n", &n);

    fgets(str, MAX_LENGTH, stdin);
    str[strlen(str) - 1] = '\0';

    printf("%s\n", str);
    printf("%d\n", n);
    return 0;
}
```

Bài thực hành 2: Đọc kí tự và chuỗi

Bài thực hành 2a. Chạy chương trình sau:

- Nhập vào chuỗi: Sunday
- Ghi lại kết quả của chương trình

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char x, y[9];
6
7      printf( "Enter a string: " );
8      scanf( "%c%s", &x, y );
9
10     printf( "The input was:\n" );
11     printf( "the character \"%c\" ", x );
12     printf( "and the string \"%s\"\n", y );
13
14     return 0;
15 }
```

Bài thực hành 2b.

- Viết chương trình nhập họ tên, MSSV, Lớp.
- In kết quả vừa nhập ra màn hình

Bài 2b

- Viết chương trình nhập họ tên, mã số sinh viên và lớp. Họ tên và lớp được lưu vào các biến xâu ký tự, còn mã số sinh viên phải lưu vào biến số nguyên. In kết quả ra màn hình, mỗi thông tin trên một dòng.

+ Input:

Tran Van An

20231234

Viet Nhat K68

+ Output:

Tran Van An

20231234

Viet Nhat K68

Định dạng đầu vào với **scanf** (3)

- Tập hợp các kí tự nằm trong cặp ngoặc vuông **[]** : theo sau dấu hiệu **'%'**
- Đọc luồng đầu vào, chỉ quan tâm tới các kí tự trong tập hợp
 - Mỗi khi gặp kí tự phù hợp, lưu vào một mảng
 - Dừng khi gặp kí tự không phù hợp
- Đọc kí tự không chứa trong tập hợp
 - Thêm dấu mũ **'^'** : **[^aeiou]**
 - Chứa các kí tự không nằm trong tập hợp

Bài thực hành 3: Bỏ qua các kí tự trong tập

- **Nhập vào chuỗi: String**
- **Ghi lại kết quả của chương trình**

```
2 #include <stdio.h>
3
4 int main()
5 {
6     char z[ 9 ] = { '\0' };
7
8     printf( "Enter a string: " );
9     scanf( "%[^aeiou]", z );
10    printf( "The input was \"%s\"\n", z );
11
12    return 0;
13}
```

Bài thực hành 4

- **Đọc và bỏ qua các kí tự từ luồng đầu vào**
- **Nhập vào chuỗi: 11-18-2000 và 11/18/2000**
- **Ghi lại kết quả của chương trình**

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int month1, day1, year1, month2, day2, year2;
6
7      printf( "Enter a date in the form mm-dd-yyyy: " );
8      scanf( "%d%c%d%c%d", &month1, &day1, &year1 );
9      printf( "month = %d  day = %d  year = %d\n\n",
10             month1, day1, year1 );
10     printf( "Enter a date in the form mm/dd/yyyy: " );
14     scanf( "%d%c%d%c%d", &month2, &day2, &year2 );
15     printf( "month = %d  day = %d  year = %d\n",
16            month2, day2, year2 );
17
18     return 0;
19 }
```

Bài thực hành 5

- Viết chương trình đọc một số nguyên và một số thực từ người dùng
- Sử dụng một biến số nguyên và một biến số thực dấu phẩy động để lưu giá trị vừa nhập
- In ra màn hình giá trị các biến đã lưu.

Bài thực hành 6

- Viết và chạy chương trình để xem giới hạn của các kiểu dữ liệu int và long.
- Mở rộng chương trình với các kiểu dữ liệu khác
- Sử dụng thư viện limits.h để viết chương trình
 - `#include <limits.h>`
 - `/* defines INT_MIN, INT_MAX, LONG_MIN, LONG_MAX, CHAR_BIT, SCHAR_MIN, SCHAR_MAX, UCHAR_MAX, CHAR_MIN, CHAR_MAX, MB_LEN_MAX, SHRT_MIN, SHRT_MAX, USHRT_MAX, INT_MIN, INT_MAX, UINT_MAX, LONG_MIN, LONG_MAX, ULONG_MAX */`

Bài thực hành 7

- Viết chương trình đọc một chuỗi từ bàn phím chỉ lấy các ký tự *aeiou*

- *Cú pháp:*

char str[10];

scanf("%[aeiou]", str);

- Lấy các ví dụ minh họa đầu vào và đầu ra của chương trình
- Gợi ý: sử dụng tập hợp kí tự với hàm scanf

Bài thực hành 8: nhập vào dữ liệu với độ rộng trường

```
#include <stdio.h>
```

```
int main( void )
```

```
{
```

```
    int x;
```

```
    int y;
```

```
    printf( "Enter a six digit integer: " );
```

```
    scanf( "%2d%d", &x, &y );
```

```
    printf( "The integers input were %d and %d\n", x, y );
```

```
    return 0; /* indicates successful termination */
```

```
}
```

- Thử nghiệm chương trình sau với 3 dữ liệu vào khác nhau. Ghi kết quả của chương trình với từng dữ liệu vào

- Mở rộng chương trình với một số kiểu dữ liệu cơ bản khác

Bài thực hành 9

- Viết chương trình yêu cầu người dùng nhập vào bán kính R .
- Sử dụng hằng số để lưu số PI
 - a) In ra diện tích và chu vi đường tròn có bán kính R vừa nhập ở trên
 - b) In ra diện tích và thể tích khối cầu có bán kính R vừa nhập ở trên

Bài thực hành 10

- Viết chương trình tính và hiển thị tiền lương của công nhân trong tuần.
- Input: Số giờ làm việc trong tuần là 40 giờ theo định mức + thời gian làm ngoài giờ.
- Thù lao một giờ làm việc theo định mức là 25000 VND và 40000 VND cho một giờ làm ngoài giờ.

Bài thực hành 11

- Viết chương trình hỏi người dùng về thông tin sách: ISBN, Tiêu đề, Giá tiền (Price), Số lượng (Qty: Quantity). Thuế VAT là 4%.
- Chương trình cần in ra các thông tin sau theo đúng giao diện:

BK Bookseller

Qty	ISBN	Title	Price	Total
-----	------	-------	-------	-------

VAT

You pay:

Bài thực hành 12

- Tìm lỗi trong đoạn chương trình sau

```
float a, b, c;  
scanf("%f", a);  
scanf("%d", &b);  
scanf("%f", &c);
```

Bài thực hành 13

- Nhận xét chức năng của chương trình sau
- Chạy chương trình với 3 dữ liệu đầu vào và ghi lại các kết quả ra tương ứng
- Thay tùy chọn "%o" trong câu lệnh scanf bằng "%i", thử nghiệm với dữ liệu nhập vào ở hệ cơ số 10, 8, 16: 018, 18, 0x18. Ghi lại kết quả thu được và nhận xét.

```
#include <stdio.h>
```

```
int main(){
```

```
    int i ;
```

```
    scanf("%o", &i);
```

```
    printf("%d", i);
```

```
    return 0;
```

```
}
```

Bài thực hành 14

Viết chương trình nhập một kí tự rồi đưa ra vị trí của ký tự này trong bảng chữ cái abc

Bài thực hành 15

Từ khóa extern

File bth15_1.c

```
// implicit definition
```

```
int GlobalVariable;
```

```
// function prototype (declaration)
```

```
void SomeFunction();
```

```
int main() {
```

```
    GlobalVariable = 1;
```

```
    SomeFunction();
```

```
    return 0;
```

```
}
```

File bth15_2.c

```
// explicit declaration
extern int GlobalVariable;
// function definition
void SomeFunction() {
    ++GlobalVariable;
}
```

- Câu 1. Biên dịch trực tiếp từ hai file mã nguồn *.c này
- Câu 2. Biên dịch hai file mã nguồn này ra các file *.o, sau đó biên dịch các file *.o ra file chương trình chạy.

limits.h

Macro	Value	Description
CHAR_BIT	8	Defines the number of bits in a byte.
SCHAR_MIN	-128	Defines the minimum value for a signed char.
SCHAR_MAX	+127	Defines the maximum value for a signed char.
UCHAR_MAX	255	Defines the maximum value for an unsigned char.

limits.h

Macro	Value	Description
CHAR_MIN	-128	Defines the minimum value for type char and its value will be equal to SCHAR_MIN if char represents negative values, otherwise zero.
CHAR_MAX	+127	Defines the value for type char and its value will be equal to SCHAR_MAX if char represents negative values, otherwise UCHAR_MAX.
MB_LEN_MAX	16	Defines the maximum number of bytes in a multi-byte character.

limits.h

Macro	Value	Description
SHRT_MIN	-32768	Defines the minimum value for a short int.
SHRT_MAX	+32767	Defines the maximum value for a short int.
USHRT_MAX	65535	Defines the maximum value for an unsigned short int.
INT_MIN	-2147483648	Defines the minimum value for an int.
INT_MAX	+2147483647	Defines the maximum value for an int.
UINT_MAX	4294967295	Defines the maximum value for an unsigned int.

limits.h

Macro	Value	Description
LONG_MIN	-9223372036854775808	Defines the minimum value for a long int.
LONG_MAX	+9223372036854775807	Defines the maximum value for a long int.
ULONG_MAX	18446744073709551615	Defines the maximum value for an unsigned long int.